03-04 Banking Stage 구현 분석

개요

Banking Stage는 TPU 파이프라인의 최종 단계로, 서명이 검증된 트랜잭션들을 실제로 실행하고 블록을 생성하여 PoH 스트림에 커밋하는 역할을 담당합니다. 이 단계는 Solana의 핵심 성능을 결정하는 가장 중요한 컴포넌트 중 하나입니다.

BankingStage 구조체 정의

```
pub struct BankingStage {
bank_thread_hdls: Vec<JoinHandle<()>>,
}
```

BankingStage는 여러 개의 스레드 핸들을 관리하며, 각 스레드는 서로 다른 역할을 수행합니다.

스레드 구성

기본 스레드 설정

```
pub const NUM_THREADS: u32 = 6;
const NUM_VOTE_PROCESSING_THREADS: u32 = 2;
const MIN_THREADS_BANKING: u32 = 1;
```

```
const MIN_TOTAL_THREADS: u32 = NUM_VOTE_PROCESSING_THREADS + MIN_THREADS_BANKING
```

스레드 역할 분담:

- **스레드 0**: Gossip 투표 트랜잭션 처리 (`solBanknStgTx00`)
- **스레드 1**: TPU 투표 트랜잭ション 처리 (`solBanknStgTx01`)
- **스레드 2-5**: 일반 트랜잭션 처리 (`solBanknStgTx02 ~ solBanknStgTx05`)

블록 생성 방법

BlockProductionMethod 열거형

```
pub enum BlockProductionMethod {
ThreadLocalMultiIterator,
CentralScheduler,
}
```

Banking Stage는 두 가지 블록 생성 방법을 지원합니다:

1. **ThreadLocalMultiIterator**: 각 스레드가 독립적으로 트랜잭션 처리
2. **CentralScheduler**: 중앙 스케줄러가 작업을 워커 스레드에 분배

핵심 컴포넌트

1. Consumer

```
pub struct Consumer {
```

```
committer: Committer,

transaction_recorder: TransactionRecorder,

qos_service: QosService,

log_messages_bytes_limit: Option<usize>,

}
```

Consumer는 트랜잭션 실행의 핵심 로직을 담당합니다.###

2. Committer

```
pub struct Committer {

transaction_status_sender: Option<TransactionStatusSender>,

replay_vote_sender: ReplayVoteSender,

prioritization_fee_cache: Arc<PrioritizationFeeCache>,

}
```

Committer는 실행된 트랜잭션을 블록에 커밋하는 역할을 담당합니다.

3. DecisionMaker

```
pub struct DecisionMaker {

// 내부 구현 세부사항

}
```

DecisionMaker는 현재 상황에 따라 트랜잭션을 처리할지, 전달할지 결정합니다.

4. Forwarder

```rust
pub struct Forwarder {
// 내부 구현 세부사항
}
```

Forwarder는 현재 노드가 리더가 아닐 때 트랜잭션을 다른 노드로 전달합니다.

메인 처리 루프

process_loop() 함수

```rust
fn process_loop(
packet_receiver: &mut PacketReceiver,
decision_maker: &DecisionMaker,
forwarder: &Forwarder,
consumer: &Consumer,
id: u32,
mut unprocessed_transaction_storage: UnprocessedTransactionStorage,
) {
let mut banking_stage_stats = BankingStageStats::new(id);
let mut tracer_packet_stats = TracerPacketStats::new(id);
let mut slot_metrics_tracker = LeaderSlotMetricsTracker::new(id);
let mut last_metrics_update = Instant::now();

loop {
```

```
// 1. 버퍼된 패킷 처리
if !unprocessed_transaction_storage.is_empty()
|| last_metrics_update.elapsed() >= SLOT_BOUNDARY_CHECK_PERIOD
{
Self::process_buffered_packets(
decision_maker,
forwarder,
consumer,
&mut unprocessed_transaction_storage,
&banking_stage_stats,
&mut slot_metrics_tracker,
&mut tracer_packet_stats,
);
last_metrics_update = Instant::now();
}


// 2. 새로운 패킷 수신 및 버퍼링
match packet_receiver.receive_and_buffer_packets(
&mut unprocessed_transaction_storage,
&mut banking_stage_stats,
&mut tracer_packet_stats,
&mut slot_metrics_tracker,
) {
```

```
Ok(()) | Err(RecvTimeoutError::Timeout) => (),
Err(RecvTimeoutError::Disconnected) => break,
}

// 3. 통계 보고
banking_stage_stats.report(1000);
tracer_packet_stats.report(1000);
}
}
```

의사결정 메커니즘

BufferedPacketsDecision 열거형

```
pub  enum BufferedPacketsDecision {
Consume(BankStart),
Forward,
ForwardAndHold,
Hold,
}
```

DecisionMaker는 현재 상황에 따라 다음 중 하나를 결정합니다:

- **Consume**: 트랜잭션을 실행하고 블록에 포함

- **Forward**: 트랜잭션을 다른 리더에게 전달
- **ForwardAndHold**: 전달하면서 동시에 버퍼에 보관
- **Hold**: 트랜잭션을 버퍼에 보관만 함## 트랜잭션

처리 파이프라인

1. 패킷 수신 및 버퍼링

```
match packet_receiver.receive_and_buffer_packets(
&mut unprocessed_transaction_storage,
&mut banking_stage_stats,
&mut tracer_packet_stats,
&mut slot_metrics_tracker,
) {
Ok(()) | Err(RecvTimeoutError::Timeout) => (),
Err(RecvTimeoutError::Disconnected) => break,
}
```

PacketReceiver는 SigVerify Stage에서 검증된 패킷을 수신하여 버퍼에 저장합니다.

2. 트랜잭션 실행

```
pub fn consume_buffered_packets(
&self,
bank_start: &BankStart,
unprocessed_transaction_storage: &mut UnprocessedTransactionStorage,
```

```
banking_stage_stats: &BankingStageStats,
slot_metrics_tracker: &mut LeaderSlotMetricsTracker,
) {
let  reached_end_of_slot = unprocessed_transaction_storage.process_packets(
bank_start.working_bank.clone(),
banking_stage_stats,
slot_metrics_tracker,
|packets_to_process, payload| {
self.do_process_packets(
bank_start,
payload,
banking_stage_stats,
&mut  consumed_buffered_packets_count,
&mut  rebuffered_packet_count,
packets_to_process,
)
},
);
}
```

Consumer는 버퍼된 트랜잭션들을 배치 단위로 처리합니다.

3. 배치 처리 최적화

```
pub const TARGET_NUM_TRANSACTIONS_PER_BATCH: usize = 64;
```

트랜잭션은 64개씩 배치로 묶어서 처리되어 효율성을 극대화합니다.

QoS (Quality of Service) 시스템

QosService 구조체

```
pub struct QosService {
// 내부 구현 세부사항
}
```

QosService는 다음과 같은 기능을 제공합니다:

1. **비용 모델**: 트랜잭션 실행 비용 예측
2. **스로틀링**: 과부하 방지를 위한 트랜잭션 제한
3. **우선순위**: 수수료 기반 트랜잭션 우선순위 결정

비용 모델 적용

```
pub struct ProcessTransactionBatchOutput {
pub(crate) cost_model_throttled_transactions_count: usize,
pub(crate) cost_model_us: u64,
pub execute_and_commit_transactions_output: ExecuteAndCommitTransactionsOutput,
}
```

비용 모델은 트랜잭션 실행 전에 리소스 사용량을 예측하여 블록 크기를 제한합니다.

트랜잭션 커밋 과정

commit_transactions() 함수

```
pub(super) fn  commit_transactions(
&self,
batch: &TransactionBatch,
loaded_transactions: &mut [TransactionLoadResult],
execution_results: Vec<TransactionExecutionResult>,
last_blockhash: Hash,
lamports_per_signature: u64,
starting_transaction_index: Option<usize>,
bank: &Arc<Bank>,
pre_balance_info: &mut PreBalanceInfo,
execute_and_commit_timings: &mut LeaderExecuteAndCommitTimings,
signature_count: u64,
executed_transactions_count: usize,
executed_non_vote_transactions_count: usize,
executed_with_successful_result_count: usize,
) -> (u64, Vec<CommitTransactionDetails>) {
// 1. 트랜잭션 커밋
let (tx_results, commit_time_us) = measure_us!(bank.commit_transactions(
```

```
batch.sanitized_transactions(),
loaded_transactions,
execution_results,
last_blockhash,
lamports_per_signature,
CommitTransactionCounts { /* ... */ },
&mut  execute_and_commit_timings.execute_timings,
));


// 2. 투표 트랜잭션 처리
bank_utils::find_and_send_votes(
batch.sanitized_transactions(),
&tx_results,
Some(&self.replay_vote_sender),
);


// 3. 잔액 수집 및 상태 전송
self.collect_balances_and_send_status_batch(
tx_results,
bank,
batch,
pre_balance_info,
starting_transaction_index,
```

```
);

// 4. 우선순위 수수료 캐시 업데이트
self.prioritization_fee_cache
.update(bank, executed_transactions.into_iter());

(commit_time_us, commit_transaction_statuses)
}
```#
중앙 스케줄러 아키텍처

CentralScheduler 방식

```rust
pub fn new_central_scheduler(
cluster_info: &Arc<ClusterInfo>,
poh_recorder: &Arc<RwLock<PohRecorder>>,
non_vote_receiver: BankingPacketReceiver,
tpu_vote_receiver: BankingPacketReceiver,
gossip_vote_receiver: BankingPacketReceiver,
num_threads: u32,
// ... 기타 매개변수들
) -> Self {
```

```
// 1. 투표 처리 스레드 생성 (기존 방식)
for (id, packet_receiver, vote_source) in [
(0, gossip_vote_receiver, VoteSource::Gossip),
(1, tpu_vote_receiver, VoteSource::Tpu),
] {
bank_thread_hdls.push(Self::spawn_thread_local_multi_iterator_thread(
id, packet_receiver, /* ... */
));
}

// 2. 워커 스레드와 스케줄러 간 통신 채널 생성
let num_workers = (num_threads).saturating_sub(NUM_VOTE_PROCESSING_THREADS);
let (work_senders, work_receivers): (Vec<Sender<_>>, Vec<Receiver<_>>) =
(0..num_workers).map(|_| unbounded()).unzip();
let (finished_work_sender, finished_work_receiver) = unbounded();

// 3. 워커 스레드 생성
for (index, work_receiver) in work_receivers.into_iter().enumerate() {
let id = (index as u32).saturating_add(NUM_VOTE_PROCESSING_THREADS);
let consume_worker = ConsumeWorker::new(
id,
work_receiver,
Consumer::new(/* ... */),
```

```
finished_work_sender.clone(),
poh_recorder.read().unwrap().new_leader_bank_notifier(),
);
bank_thread_hdls.push(
Builder::new()
.name(format!("solCoWorker{id:02}"))
.spawn(move || { consume_worker.run(); })
.unwrap(),
);
}

// 4. 중앙 스케줄러 스레드 생성
let  scheduler_controller = SchedulerController::new(
decision_maker,
PacketDeserializer::new(non_vote_receiver, bank_forks),
bank_forks,
PrioGraphScheduler::new(work_senders, finished_work_receiver),
worker_metrics,
);
bank_thread_hdls.push(
Builder::new()
.name("solBnkTxSched".to_string())
.spawn(move || { scheduler_controller.run(); })
```

```
.unwrap()
);
}
```

스케줄러 구성 요소

1. **PacketDeserializer**: 패킷을 트랜잭션으로 역직렬화
2. **PrioGraphScheduler**: 의존성 그래프 기반 스케줄링
3. **ConsumeWorker**: 실제 트랜잭션 실행을 담당하는 워커
4. **SchedulerController**: 전체 스케줄링 프로세스 제어

성능 통계 및 모니터링

BankingStageStats 구조체

```
pub struct BankingStageStats {
last_report: AtomicInterval,
id: u32,
receive_and_buffer_packets_count: AtomicUsize,
dropped_packets_count: AtomicUsize,
dropped_duplicated_packets_count: AtomicUsize,
dropped_forward_packets_count: AtomicUsize,
newly_buffered_packets_count: AtomicUsize,
newly_buffered_forwarded_packets_count: AtomicUsize,
```

```
current_buffered_packets_count: AtomicUsize,
rebuffered_packets_count: AtomicUsize,
consumed_buffered_packets_count: AtomicUsize,
forwarded_transaction_count: AtomicUsize,
forwarded_vote_count: AtomicUsize,
// 타이밍 메트릭
consume_buffered_packets_elapsed: AtomicU64,
receive_and_buffer_packets_elapsed: AtomicU64,
filter_pending_packets_elapsed: AtomicU64,
packet_conversion_elapsed: AtomicU64,
transaction_processing_elapsed: AtomicU64,
}
```

수집되는 주요 메트릭:

- **패킷 처리량**: 수신, 버퍼링, 소비된 패킷 수
- **드롭 통계**: 중복, 전달, 일반 드롭된 패킷 수
- **처리 시간**: 각 단계별 소요 시간
- **전달 통계**: 다른 노드로 전달된 트랜잭션 수

실시간 보고

```
fn report(&mut self, report_interval_ms: u64) {
if self.last_report.should_update(report_interval_ms) {
datapoint_info!(
```

```
"banking_stage-loop-stats",
("id", self.id, i64),
("receive_and_buffer_packets_count", /* ... */),
("dropped_packets_count", /* ... */),
// ... 기타 메트릭들
);
}
}
```

1초마다 상세한 성능 메트릭을 보고합니다.

오류 처리 및 재시도 메커니즘

재시도 가능한 트랜잭션

```
pub struct ExecuteAndCommitTransactionsOutput {
pub(crate) retryable_transaction_indexes: Vec<usize>,
pub commit_transactions_result: Result<Vec<CommitTransactionDetails>, PohRecord
// ... 기타 필드들
}
```

다음과 같은 경우 트랜잭션이 재시도됩니다:

1. **일시적 리소스 부족**: 계정 잠금 충돌
2. **블록 경계**: 현재 블록이 가득 참

3. **PoH 높이 제한**: 최대 PoH 높이 도달

트랜잭션 상태

```
pub  enum CommitTransactionDetails {
Committed { compute_units: u64 },
NotCommitted,
}
```

각 트랜잭션의 최종 상태를 추적하여 적절한 후속 처리를 수행합니다.## 투표 트랜잭션 특별 처리

투표 전용 스레드

```
// 스레드 0: Gossip 투표
(
gossip_vote_receiver.clone(),
UnprocessedTransactionStorage::new_vote_storage(
latest_unprocessed_votes.clone(),
VoteSource::Gossip,
),
)

// 스레드 1: TPU 투표
(
```

```
tpu_vote_receiver.clone(),
UnprocessedTransactionStorage::new_vote_storage(
latest_unprocessed_votes.clone(),
VoteSource::Tpu,
),
)
```

투표 트랜잭션은 합의 과정에 중요하므로 별도의 전용 스레드에서 처리됩니다.

LatestUnprocessedVotes

```
let latest_unprocessed_votes = Arc::new(LatestUnprocessedVotes::new());
```

중복된 투표 트랜잭션을 효율적으로 관리하기 위한 특별한 저장소입니다.

메모리 관리 및 최적화

버퍼 크기 제한

```
const TOTAL_BUFFERED_PACKETS: usize = 700_000;
```

```
let batch_limit = TOTAL_BUFFERED_PACKETS /
((num_threads - NUM_VOTE_PROCESSING_THREADS) as usize);
```

전체 메모리 사용량을 제한하기 위해 스레드별로 버퍼 크기를 분할합니다.

데이터 예산 관리

```
let  data_budget = Arc::new(DataBudget::default());
```

네트워크 대역폭 사용량을 제어하기 위한 데이터 예산 시스템을 사용합니다.

슬롯 경계 처리

슬롯 전환 감지

```
const SLOT_BOUNDARY_CHECK_PERIOD: Duration = Duration::from_millis(10);
```

```
if !unprocessed_transaction_storage.is_empty()
|| last_metrics_update.elapsed() >= SLOT_BOUNDARY_CHECK_PERIOD
{
// 슬롯 경계 확인 및 처리
}
```

10ms마다 슬롯 경계를 확인하여 적절한 시점에 블록을 마무리합니다.

리더 슬롯 메트릭

```
let  metrics_action = slot_metrics_tracker.check_leader_slot_boundary(
decision.bank_start(),
Some(unprocessed_transaction_storage),
```

```
);
```

리더 슬롯의 시작과 끝을 추적하여 성능 메트릭을 수집합니다.

전달 메커니즘 (Forwarding)

ForwardOption 열거형

```
pub  enum ForwardOption {
NotForward,
ForwardTpuVote,
ForwardTransaction,
}
```

현재 노드가 리더가 아닐 때 트랜잭션을 적절한 리더에게 전달합니다.

전달 조건

1. **리더십 상태**: 현재 노드가 리더가 아님
2. **슬롯 오프셋**: 미래 슬롯의 리더에게 전달
3. **네트워크 상태**: 연결 가능한 리더 존재

요약

Banking Stage는 Solana TPU의 최종 단계로서 다음과 같은 핵심 기능을 제공합니다:

1. **고성능 트랜잭션 실행**: 다중 스레드를 통한 병렬 처리
2. **지능적 스케줄링**: 중앙 스케줄러 또는 스레드 로컬 방식 선택

3. **QoS 시스템**: 비용 모델 기반 리소스 관리

4. **효율적인 커밋**: 배치 단위 트랜잭션 커밋

5. **투표 최적화**: 합의 트랜잭션 전용 처리 경로

6. **실시간 모니터링**: 상세한 성능 메트릭 수집

7. **적응적 전달**: 네트워크 상황에 따른 트랜잭션 라우팅

8. **메모리 최적화**: 버퍼 크기 제한 및 데이터 예산 관리

이러한 설계를 통해 Banking Stage는 높은 처리량과 낮은 지연시간을 달성하면서도 네트워크 안정성과 합의 무결성을 보장합니다. 특히 투표 트랜잭션의 특별 처리와 중앙 스케줄러 옵션을 통해 다양한 네트워크 조건에서 최적의 성능을 제공할 수 있습니다.